



Day 1 – Claude AI & AI-Assisted Development

Building Modern .NET Applications with Claude

.NET

ASP.NET CORE

C#

CLAUDE

XUNIT

From AI Chat to AI-Assisted Engineering

Most developers start by asking Claude questions. AI-assisted engineering goes further — Claude becomes an **active participant across every phase** of the software development lifecycle.



Understand

Analyse codebases, architecture, and domain concepts



Review

Identify bugs, security issues, and code quality problems



Document

Write inline comments, API docs, and changelogs



Design

Propose patterns, evaluate trade-offs, and plan implementation



Test

Create test matrices, unit tests, and edge case coverage



Code

Generate, complete, and modify production-quality code



Debug

Diagnose errors, trace root causes, and suggest fixes

"The shift is from Claude as a search engine to Claude as a collaborative engineering partner."

Learning Objectives

By the end of Day 1, participants will be able to:

- 1 Understand the Claude platform and model ecosystem**
- 2 Select appropriate models for development tasks**
- 3 Use Claude APIs securely and manage tokens & context windows**
- 4 Write effective prompts for software engineering**
- 5 Apply Claude Code, Tool Use, MCP, RAG and agentic workflows**

Day 1 Agenda

Module 1

Claude Platform & Model Overview

Module 2

Prompt Engineering for Developers

Module 3

Claude Code & Developer Productivity

Module 4

Tool Use, MCP & Agentic Development

📄 Throughout the day we use a **.NET Order Management application** as the common development scenario.

Training Scenario: Enterprise Order Management API

Technology Stack

- ASP.NET Core & C#
- Entity Framework Core
- SQL Server & REST APIs
- xUnit & Git

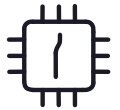
Domain Flow

Customer → Order → Order Items → Inventory → Payment

Business Requirements — An order must:

1. Validate customer
2. Validate items
3. Check inventory
4. Calculate total
5. Apply business rules
6. Save order
7. Return order number

Claude Platform & Model Overview



Models & Selection

Claude model family, selection criteria, cost and performance



Platform & Auth

Console, API authentication, key management



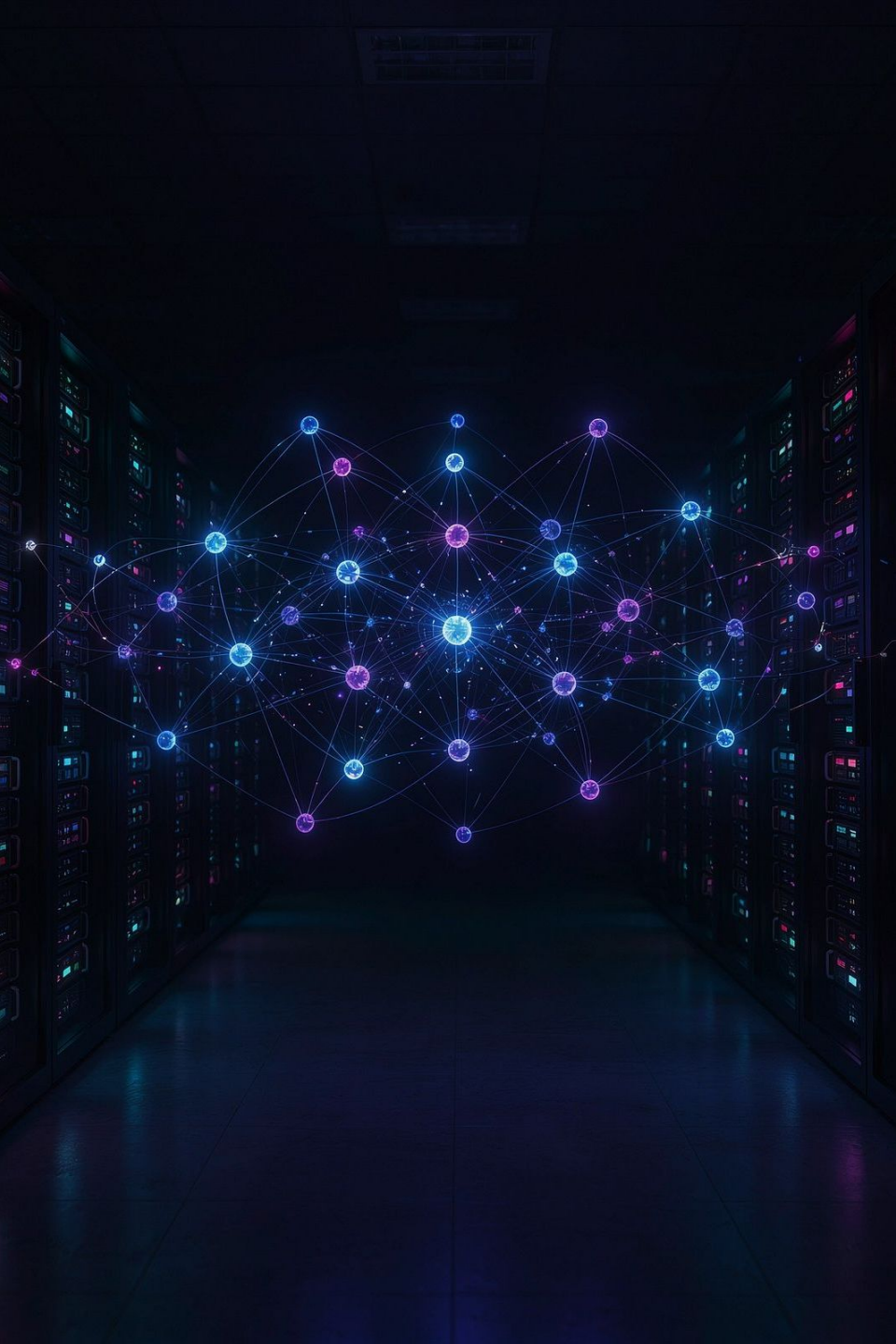
Context & Tokens

Context windows, large repository context, limitations



SDK & Tools

SDK overview, Fable, Cowork



What is Claude?

Claude is an AI platform that supports developers across the full engineering workflow:

Code

Explanation, generation, refactoring

Quality

Debugging, review, test generation

Architecture

Analysis, documentation, repository understanding

Automation

Tool interaction, agentic workflows

Claude Development Ecosystem

Claude Chat

Technical questions, code explanation, architecture discussions, debugging

Claude Platform / API

AI-enabled applications, enterprise automation, custom AI services

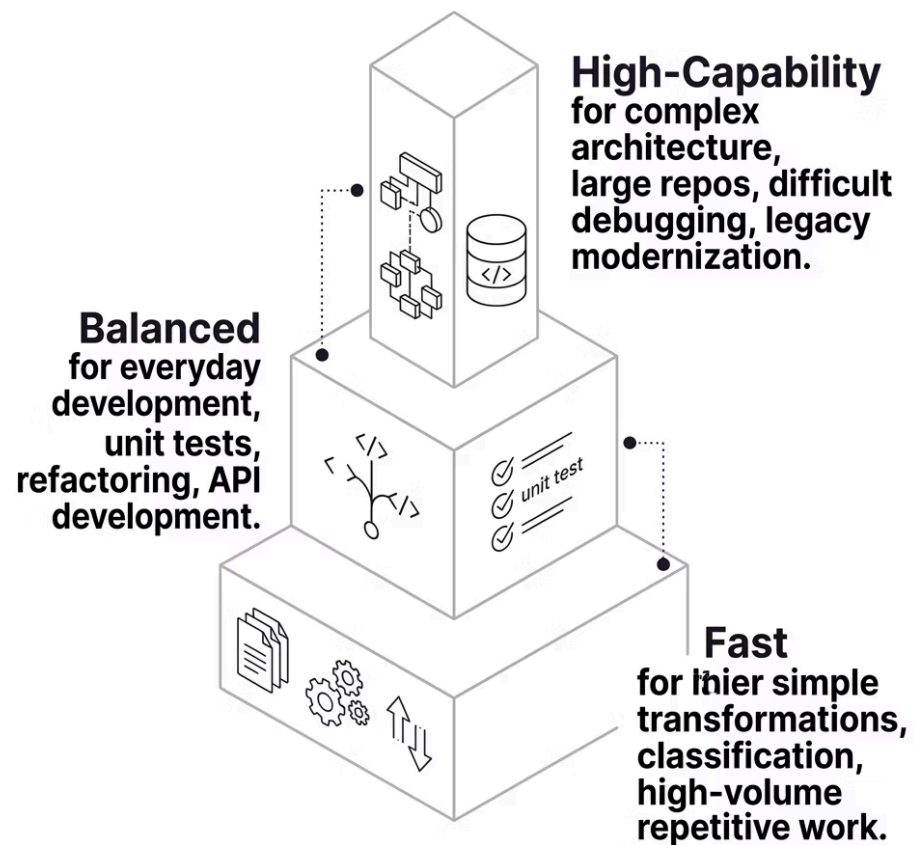
Claude Code

Repository analysis, code modification, testing, refactoring

MCP / Tool Use

APIs, databases, enterprise systems, development tools

Claude Model Family



What varies across models

- Intelligence & reasoning depth
- Coding capability
- Speed & latency
- Context window size
- Cost per token

☐ Match the model tier to the task — not every task needs the most powerful model.

Model Selection for Development Tasks

Simple Task

"Generate a C# DTO." — Small context, low reasoning, fast response needed.

→ **Cost-efficient model**

Standard Task

"Implement and test an ASP.NET Core service." — Moderate reasoning, code generation, validation.

→ **Balanced model**

Complex Task

"Analyze a large legacy application and design a modernization strategy." — Large context, architecture reasoning.

→ **High-capability model**

Model Selection Considerations



Complexity & Accuracy

How difficult is the problem? How critical is correctness?



Context Size

How much code or information is needed?



Latency & Cost

How quickly is a response required? How many requests will be processed?



Agentic Behaviour

Does Claude need multiple reasoning and tool-use steps?



Do not automatically use the largest model for every task.

Claude Platform / Console



The Claude platform enables developers to:

- Access models
- Manage API keys
- Experiment with prompts
- Monitor usage
- Integrate Claude into applications

Typical Flow

Developer → Claude Platform → API Key → Claude API → .NET Application

API Authentication & Key Management

Never

- Hard-code keys
- Commit keys to Git
- Add keys to config files in source control
- Print keys in logs

Local Development

- Environment variables
- .NET User Secrets

Production

- Azure Key Vault
- AWS Secrets Manager
- Enterprise secret-management platform



Treat an AI API key like a database password.

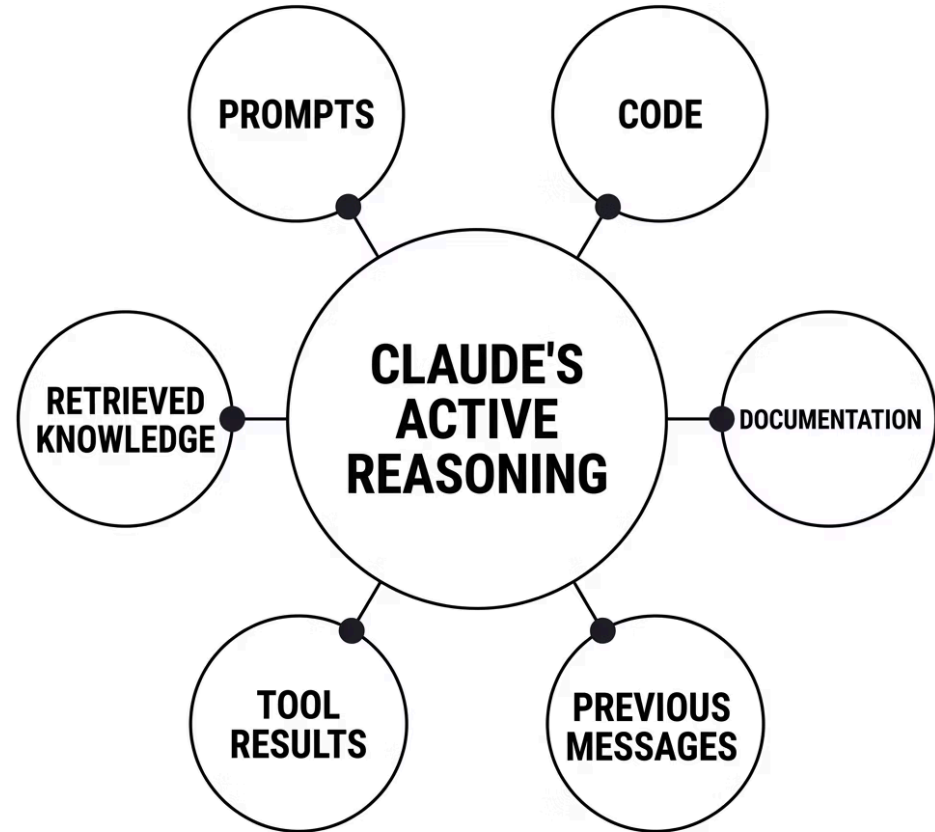
Tokens & Context Windows

What are tokens?

Claude processes text as tokens — representing words, code, symbols, documentation, and conversation history.

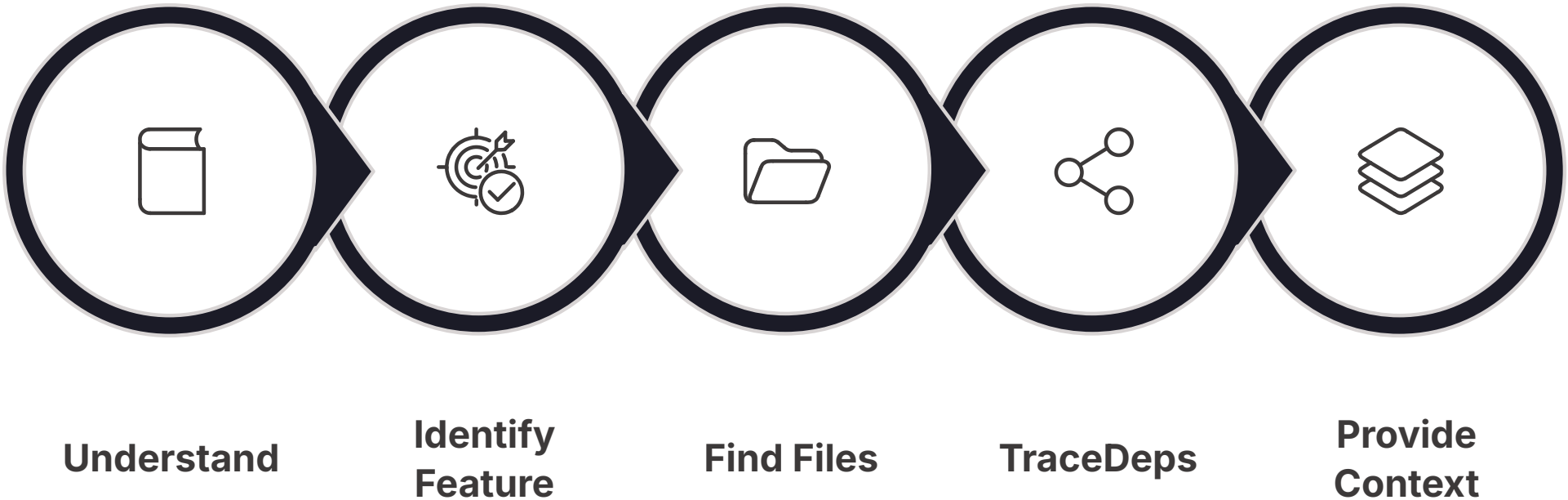
What is the context window?

The information Claude can currently consider: prompts, code, documentation, previous messages, tool results, and retrieved knowledge.



Context Management for Large Repositories

Do not send an entire enterprise repository unless required.



📌 **Relevant context is more valuable than excessive context.**

Cost & Performance Considerations

Poor Workflow

- Huge Context
- Vague Prompt
- Incorrect Result
- Repeated Attempts

Better Workflow

- Discover
- Scope
- Plan
- Implement
- Validate

Claude Limitations & Validation

Claude can generate useful code, but AI-generated output may contain:

Logic Errors

Incorrect business logic or incorrect assumptions

Hallucinated APIs

Non-existent methods or libraries

Security Gaps

Vulnerabilities or missing edge cases

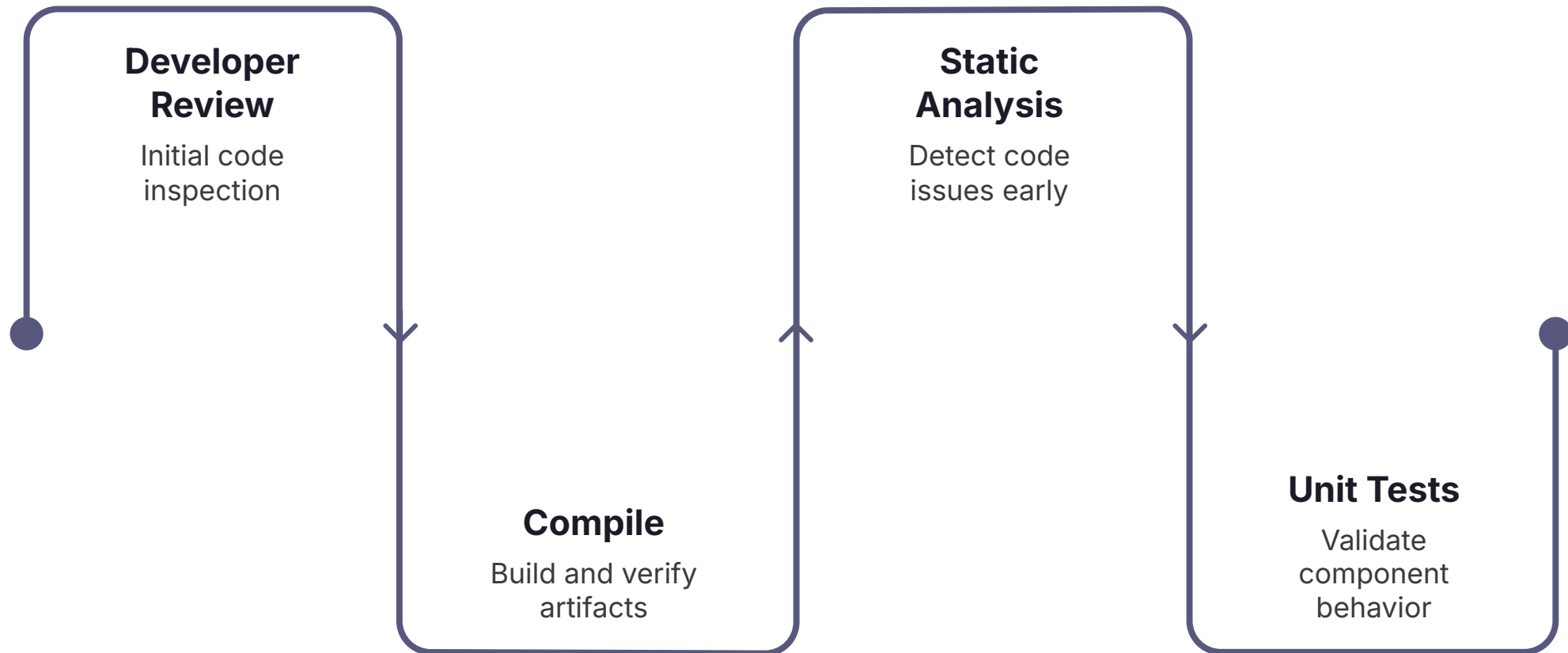
Quality Issues

Performance problems, unnecessary complexity, compatibility issues

📄 **AI-generated code is a proposal until validated.**



AI Validation Pipeline



AI improves productivity. It does not replace engineering controls.

Claude SDK Overview



Architecture

.NET Application → Claude Client / SDK → Claude API → Claude Model → Response

Common Use Cases

- AI assistants & developer tools
- Document and code analysis
- Workflow automation
- Enterprise knowledge systems

Fable & Cowork Overview

Fable

Suitable for advanced workloads:

- Complex codebase analysis
- Architecture reasoning
- Legacy modernization
- Difficult debugging & refactoring

Cowork

Designed for multi-step knowledge work:

- Documents & files
- Planning & research
- Connected applications
- Synthesis across sources

Claude Code Repository Work	Cowork Knowledge Work
---	---

DEMO 1

Configure Claude & Perform Code Explanation / Generation

01

Explain an existing C# method

02

Identify hidden business rules

03

Find maintainability issues

04

Recommend improvements

05

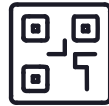
Generate an improved version

Prompt Engineering for Developers



Prompt Fundamentals

System vs user prompts, structured prompting, context-aware prompting, few-shot prompting



Development Tasks

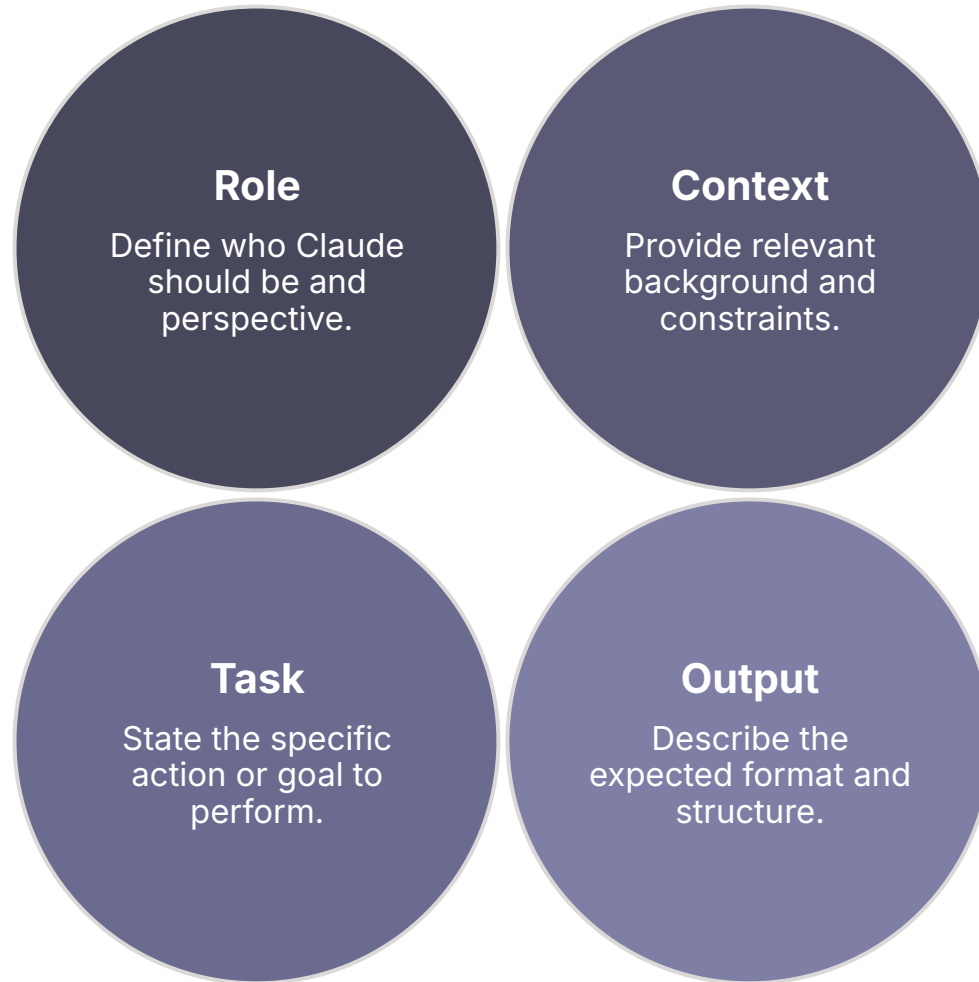
Code generation, debugging, code review, testing



Advanced Analysis

Legacy analysis, architecture review, security review, refactoring

The Developer Prompt Pattern



System Prompt vs User Prompt

System Prompt

Defines **ongoing behaviour**.

"You are a senior .NET developer. Prefer maintainable solutions, consider security and preserve public APIs."

= Operating Rules

User Prompt

Defines the **immediate task**.

"Review CreateOrderAsync for concurrency issues."

= Current Task

Structured & Context-Aware Prompting

Weak Prompt

"Optimize this query."

Better Prompt

"This query executes through EF Core against SQL Server. The Orders table contains 15 million rows and only the latest 100 orders are needed."

Structured Prompt

- **Role:** Senior .NET developer
- **Context:** ASP.NET Core + EF Core + SQL Server
- **Task:** Review the query
- **Constraints:** Do not change business behaviour
- **Expected Output:** Findings + recommendations



Context reduces assumptions.

Few-Shot Prompting

Provide examples of the expected behaviour before asking Claude to perform the task.

Example Input

```
repository.Get(id).Result
```

Expected Review Format

Severity: High

Issue: Blocking **async** operation

Recommendation: Use **async/await**

Then Ask

"Review the following code using the same format."

- ❏ Examples improve output consistency and reduce ambiguity in Claude's responses.

Prompting for Code Generation, Debugging & Review

Code Generation

Specify architecture, framework, requirements, constraints and expected components.

Debugging

Provide observed behaviour, error, environment, expected behaviour and diagnostic goal.

Code Review

Ask Claude to check correctness, security, performance, maintainability, async usage and testability.

Prompting for Testing

Start with a test matrix, not "Write tests."

- Happy path & boundary cases
- Invalid & null input
- Dependency failures & timeouts
- Cancellation & duplicate requests
- Concurrency & authorization

Generate automated tests **from the approved matrix.**



Advanced Developer Prompts

Legacy Code Analysis

Understand responsibilities and dependencies before refactoring.

Architecture Review

Evaluate coupling, scalability, resilience and maintainability.

Security Review

Evaluate authentication, authorization, validation and data exposure.

Refactoring

Improve code while preserving behaviour and public contracts.

📋 Understand → Evaluate → Plan → Change

Prompt Engineering for Software Developers

Create prompts for analyzing, generating, improving and testing .NET application code using the pattern:

Role

Context

Task

Constraints

Expected Output

Claude Code & Developer Productivity

Claude Code brings AI assistance directly into **repository-level** software development.



Understand

Repository understanding, code search



Validate

Test generation, build and test execution



Build

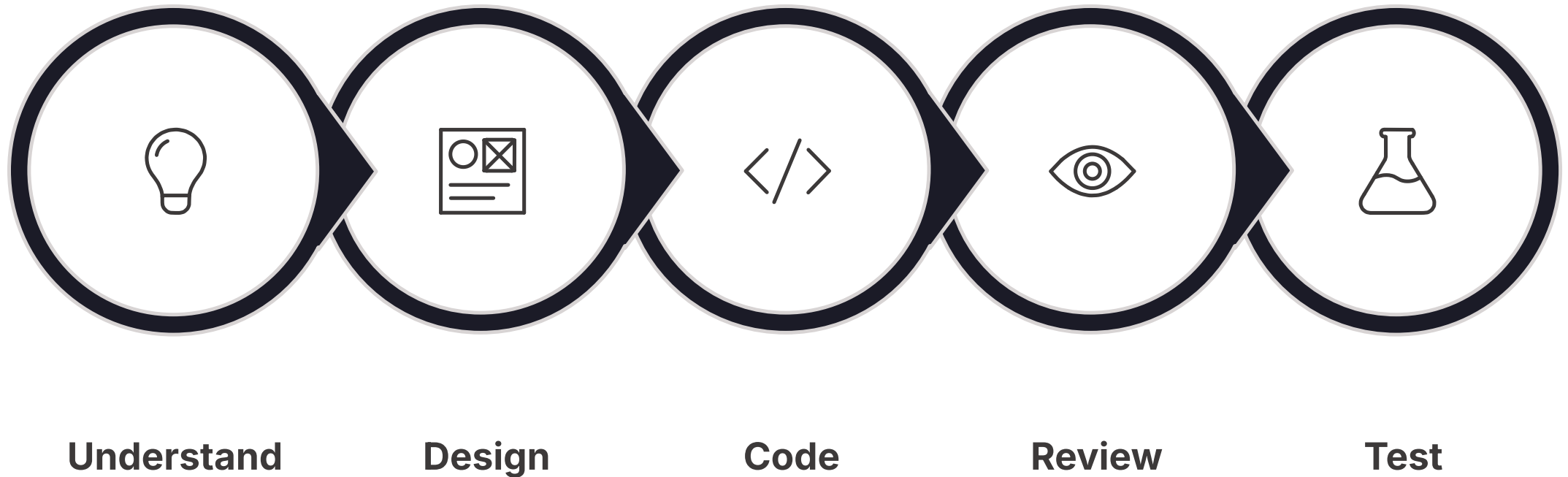
Code generation, modification, refactoring



Deliver

Documentation, pull request support

Claude Across the Development Lifecycle



Claude is not only a code generator. It can assist throughout the entire software development lifecycle.

Understanding an Existing Codebase

Before making changes, ask Claude to understand:

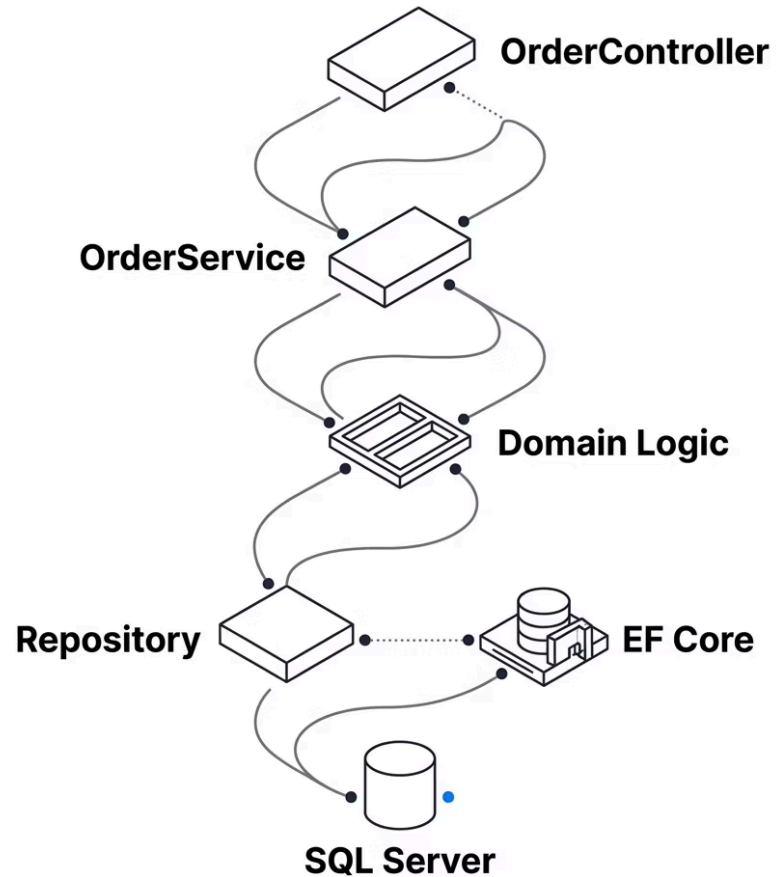
- Solution structure & projects
- Architecture & domain concepts
- Dependencies & database access
- Existing tests & application flow

"Trace POST /api/orders from the controller through persistence."



📄 **Understand before changing.**

Repository-Level Context

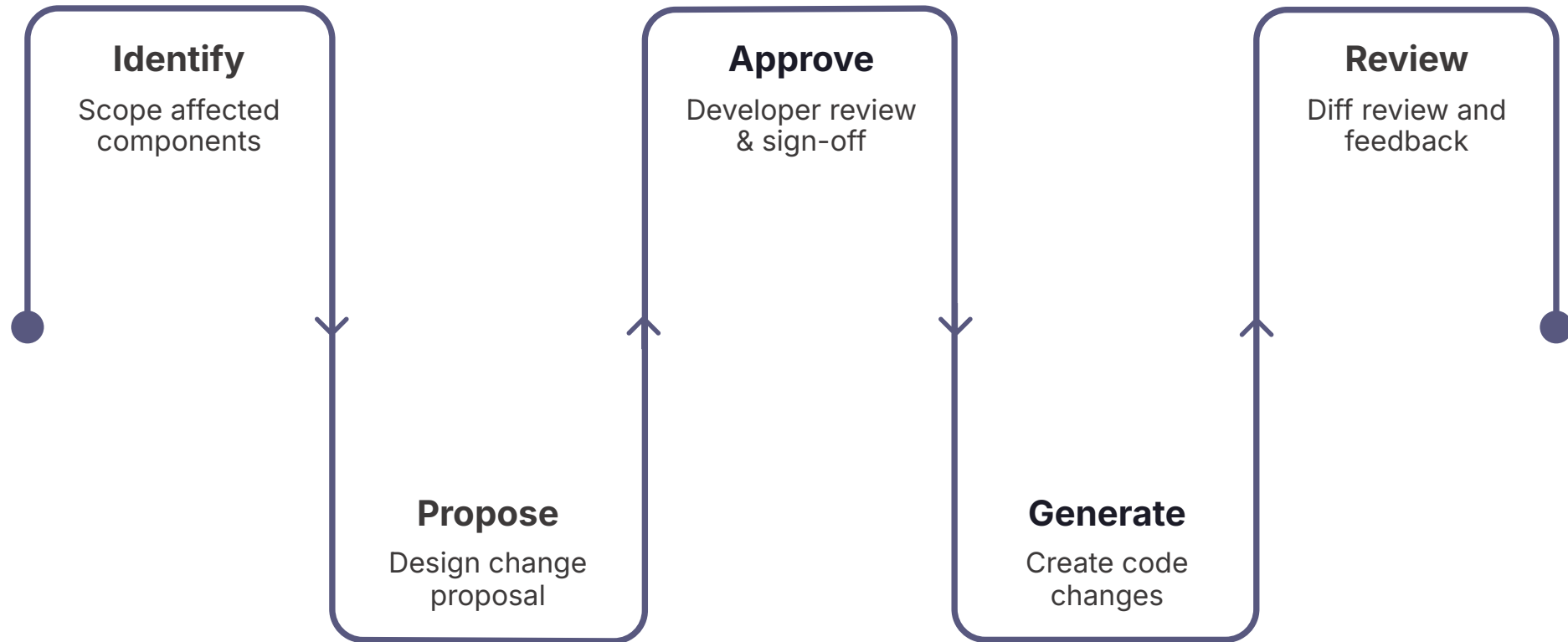


Claude Code can reason across:

- Multiple projects & classes
- Interfaces & configuration
- Tests & infrastructure
- Documentation

📄 Developers can understand unfamiliar applications significantly faster.

Code Generation & Modification



**Avoid unrelated
modifications**

Preserve contracts

**Follow project
architecture**

**Follow coding
standards**

Refactoring & Code Explanation

Claude can identify:

- Long methods & duplicate code
- Tight coupling & poor naming
- Excessive responsibilities
- Hard-to-test logic

Recommended Workflow

01

Analyze

02

Add Tests

03

Propose Refactoring

04

Refactor

05

Run Tests

Goal: **Improve design without changing behaviour.**

Reusable Development Instructions

Teams can define reusable project rules for consistent AI-assisted development.

Architecture

Clean Architecture, thin controllers, dependency inversion

Coding

Async APIs, CancellationToken, nullable reference types

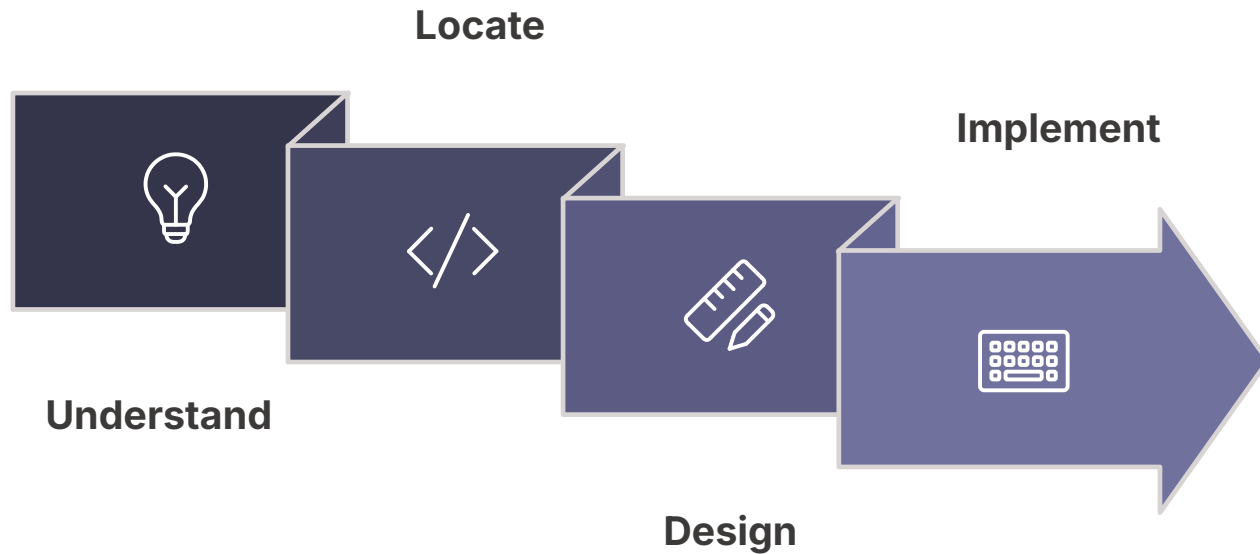
Testing

xUnit, Moq, Arrange / Act / Assert

Security

Never log secrets, protect sensitive data

Feature Development & PR Assistance



Claude can generate PR content:

- PR title & summary
- Technical changes description
- Tests & risks
- Reviewer checklist

Code Review Support

Claude can review changes for:

- Correctness & security
- Performance & async behaviour
- Nullability & dependency injection
- Error handling & backward compatibility
- Test coverage



📄 Ask Claude to **show findings first**, then decide which changes to implement.



LAB 2

Claude Code – Existing Project Enhancement

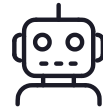
Use Claude Code to understand an existing .NET project, implement a small change, refactor code, generate documentation and create initial unit tests.

Tool Use, MCP & Agentic Development



Tool Use

Function calling, tool schemas, live information retrieval



Agentic Workflows

Goal-driven multi-step reasoning and action



MCP

Standardized protocol for enterprise system integration



RAG

Enterprise knowledge integration and grounded responses

Tool Use & Function Calling

The Problem

Claude may need live information not available in the model.

"How much inventory exists for P1001?"

Claude should **not guess** — it should call a tool.

Heading font:
Inter. The
Quick Brown
Fox Jumps
Over The
Lazy Dog.

Body font: Inter. Use it for
paragraphs and smaller
text.

ABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz
0123456789.



📄 Use tools for live or authoritative information.

Tool Schemas

A tool schema tells Claude:

- Tool name & purpose
- Input parameters
- Required fields
- Expected behaviour

Example Schema

- **Tool:** get_inventory
- **Description:** Returns current stock for a product
- **Input:** productId

📄 Clear names and narrow responsibilities improve agent reliability.

Agentic Development

Traditional AI

Prompt → Response

Agentic AI

Goal → Reason → Tool → Result → Next Action

Example: Can an order be fulfilled?

01

Reason: Need order and inventory information

02

Tool: Get order → Products identified

03

Tool: Check inventory

04

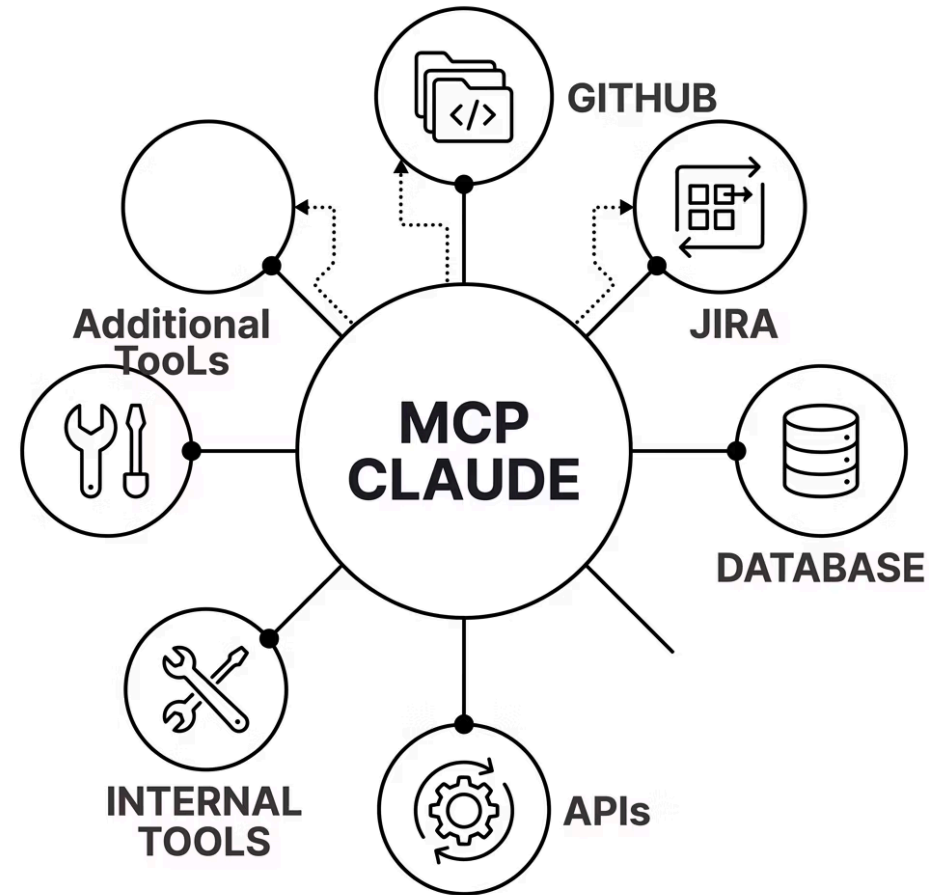
Action: Determine fulfillment

MCP Fundamentals

Model Context Protocol

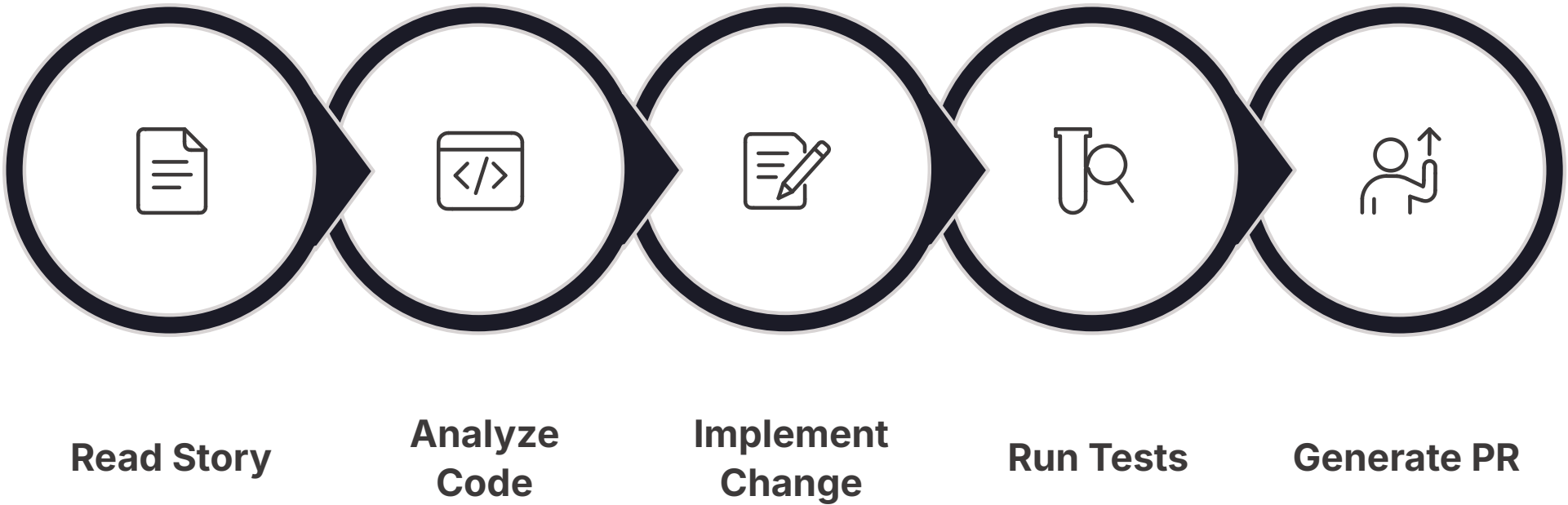
A standardized method for connecting AI systems with external systems and tools.

- APIs & Databases
- Development tools
- Files & enterprise systems
- Internal services



Enterprise MCP Scenario

Developer asks: *"Implement Jira story ORD-245."*



☐ Access control and human approval remain essential.

RAG & Enterprise Knowledge Integration

How RAG works

Claude retrieves trusted enterprise knowledge before responding.

"What is our refund policy?"

Claude → Search Knowledge Base → Retrieve Policy →
Generate Grounded Answer

Enterprise Sources

- Policies & architecture documents
- API documentation
- Coding standards
- Product manuals

RAG vs MCP

RAG = Knowledge

What information should Claude know?

Search documentation · Retrieve policy · Retrieve architecture standard

MCP = Tools & Actions

What system should Claude interact with?

Query Jira · Check inventory · Create ticket · Trigger pipeline

Tool Use, MCP & Agentic Workflow

Build an agentic workflow where Claude reasons across multiple steps, calls external tools, and produces a structured outcome.

01	02	03
Goal	Reason	Tool Call
Define the objective — e.g. <i>"Can order ORD-1042 be fulfilled?"</i>	Claude analyses available tools and determines what information is needed	Claude invokes <code>get_inventory</code> , <code>get_order</code> , or <code>check_supplier</code> via MCP
04	05	
Result	Next Action	
Claude receives structured data and evaluates the response	Claude either calls another tool or returns a final answer	

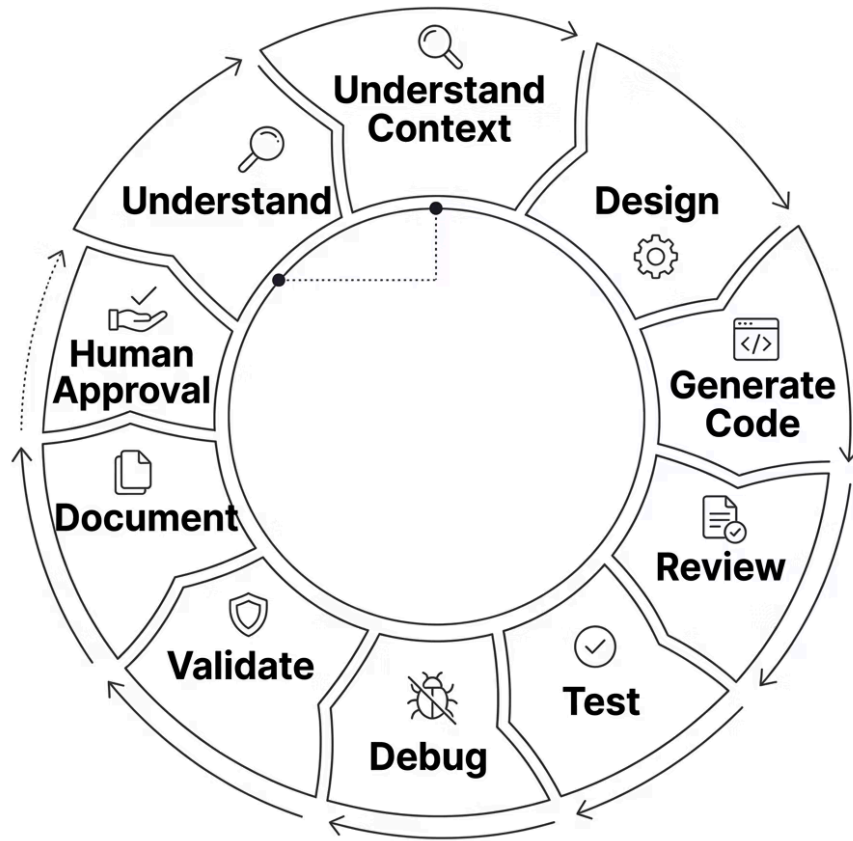
Exercise Tasks

- Configure an MCP server with at least two tools
- Write tool schemas with clear descriptions and required parameters
- Prompt Claude with a multi-step business question
- Observe Claude's reasoning and tool selection
- Validate the final output against expected results

Success Criteria

- Claude selects the correct tool without being told which to use
- Tool inputs are correctly formed
- Claude handles a missing or partial result gracefully
- Final answer is accurate and well-reasoned

Day 1 Integrated View



Supported by

- Claude Models
- Prompt Engineering
- Claude Code
- Tool Use & MCP
- RAG
- Agentic Workflows

Traditional AI vs AI-Assisted Engineering

Traditional Usage

Developer → Prompt → Generated Code → Copy / Paste

AI-Assisted Engineering

Requirement → Context → Plan → Implementation →
Review → Testing → Validation → Documentation

AI becomes part of the engineering workflow rather than only a code generator.

Day 1 Key Takeaways

Model Selection

Match capability to task complexity.

Context Engineering

Give Claude the right information, not simply more information.

Prompt Engineering

Role → Context → Task → Constraints → Expected Output

Validation

Never treat generated code as automatically production-ready.

Agentic AI

Goal → Reason → Tool → Result → Next Action

MCP & RAG

MCP connects tools. RAG connects knowledge.



AI Should Augment Engineering Discipline

Not the objective

"Let AI write all our code."

The objective

"Use AI intelligently across the software development lifecycle while maintaining architecture, security, testing, governance and human accountability."

Day 1 Outcome

Participants now understand how Claude can move from:



AI Assistant

Answering technical questions



Coding Assistant

Generating and reviewing code



Repository Agent

Understanding and modifying entire codebases



Tool-Using Enterprise AI

Connecting to APIs, databases and enterprise systems